

Agency Blog SaaS: Demo

Brief

A short, accurate account of what this project is, what it proves, and where it falls short.

The framing, first, because everything depends on it

This repository is **not** the product. It is a static web page that draws pictures of a product. The real system, **agency-blog-saas**, is a production Django SaaS behind `digitalise.agency` with real clients, real billing and real AI generation. Its source is private and stays private: it is the agency's live product, not a codebase that could be scrubbed and released. This demo exists so the product's screens can be clicked by someone who will never be shown the real repository.

Nearly everything on the page is **fabricated**: the credit balance, the client, the updates, the generation itself. Nothing is computed, nothing is sent anywhere. The one exception is the blog library, which reproduces a real client's already-public articles, and the page labels that distinction in four separate ways.

1 What it is

One HTML page, one CSS file, and two JavaScript files totalling 216 lines of behaviour plus 150 KB of stored data. No framework, no build step, no backend, no dependencies, no secrets, and no network calls at runtime. Deployment is two lines of `netlify.toml` saying "publish the folder as it is". To run it, open `index.html`.

Four tabs mirror the real product's screens, and a fifth hidden panel reads an article in place:

- **AI Blog Generator**: a form, an Autofill button, and a Generate button that runs a fixed 2.9-second sequence of status messages, subtracts 100 from a number, and produces a post that was already sitting in the page.
- **Blog Library**: fifteen real, live articles from a real client's public blog.
- **Subscriptions**: three real, published price tiers and a redemption box that always answers that it is a demo.
- **Client Updates**: three lines for an openly invented company.
- **Reader**: opens any card into a full article with hero image, prose, tables, citations, references, and a link back to the original.

2 The idea that makes it worth reading

The interesting problem here is not technical. A fabricated demo is a fiction, and the whole design question is how to keep a visitor from mistaking the fiction for the fact.

The project's answer is that data on the page comes in three tiers, and each is labelled by a different mechanism.

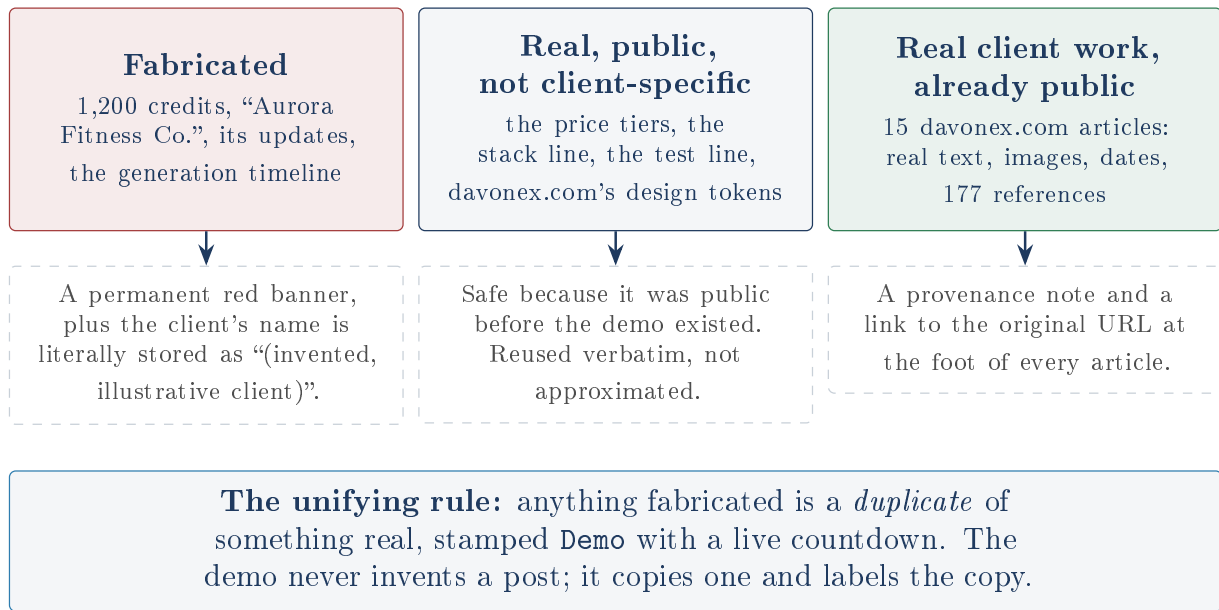


Figure 1: Three provenances on one page, and the machinery that keeps them apart.

Shipping a real client's genuine, shipped work beside clearly-labelled fakes is an unusual choice and a defensible one: the visitor sees real output and a working interaction in one place, with a checkable link on every article.

3 Architecture

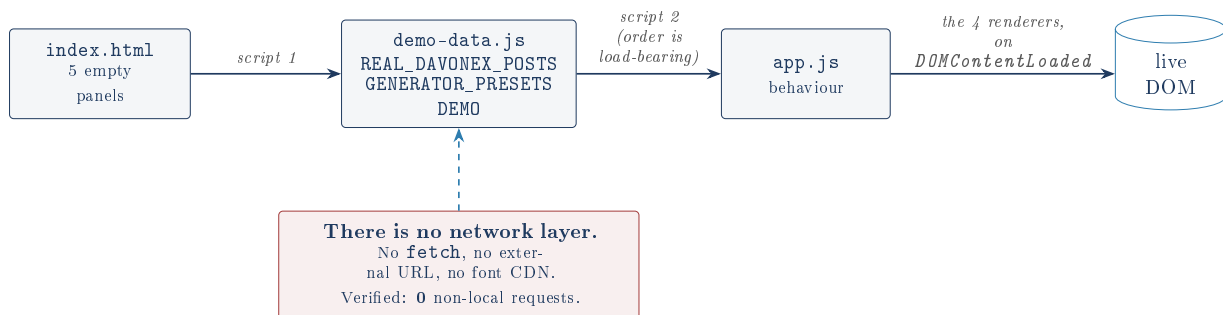


Figure 2: The complete architecture. Two script tags, three globals, four render functions.

Each file has exactly one job: HTML is structure, **demo-data.js** is data, **app.js** is behaviour, **styles.css** is appearance. Because both scripts are plain (non-module) scripts, their top-level constants share one global scope, and **app.js** reads **demo-data.js**'s constants by name. That makes the order of two lines in the HTML a real dependency that neither file records.

The one design decision worth defending is that `GENERATOR_PRESETS` is *derived* from the posts rather than authored:

```
1 const GENERATOR_PRESETS = REAL_DAVONEX_POSTS.map(p => ({
2   slug: p.slug, topic: p.title, description: p.excerpt,
3   keywords: p.title.toLowerCase().replace(/[:]/g, " ")
4     .split(" ").filter(w => w.length > 3).slice(0, 6).join(", ")
5 }));
```

Why that matters

If Autofill wrote one thing into the form and Generate produced something unrelated, the demo would be quietly lying about the one property a generator must have: that the output follows from the input. Deriving each preset *from* the post it will produce makes that link structural rather than clerical. Preset *i* carries post *i*'s slug; Autofill records it; Generate looks it up. They cannot drift apart because there is nothing to keep in sync. (The README adds that this mirrors the real pipeline's deterministic-per-brief behaviour. That is a claim about the private system and cannot be checked from this repository.)

Two other techniques are worth naming. **Event delegation**: the library's cards are destroyed and rebuilt every second by a countdown, so no listener is attached to a card; one listener sits on the container and uses `e.target.closest("[data-open-post]")` to find which card was hit. **Single-slot state**: `libraryDemo` is one object or `null`, never a list, so generating twice replaces rather than accumulates.

4 Verified, not assumed

The page was driven in a real headless Chrome and the data evaluated in Node.

Claim	Result
No network calls at runtime	True : 0 non-local requests
No JavaScript errors across the full tour	True : 0 errors
15 real posts, unique slugs and titles, images on disk	True
Generate adds one card, credits 1,200 to 1,100	True
The autofilled post is the generated post	True
Article renders with prose, table, 11 references	True
Citation links resolve	False : 21 links, 21 dead
A typed topic is respected	False : returns an unrelated post
Credits have a floor	False : renders -50
The ticker stops when the card expires	False : runs forever
132/132 tests, 56% coverage	Not verifiable here
CSS tokens copied 1:1 from davonex.com	Not verifiable here

5 What does not hold up

The banner's central claim is false as written

The red banner says "*nothing shown is a real client's data*", and the footer repeats it. But the Blog Library is fifteen real posts from a real client, and the project's own README, CREDITS and provenance notes say so proudly.

The intended meaning is obviously "*no private client data*", and everything shown is genuinely public. But as written, the page's most prominent honesty statement contradicts the page's most interesting honest feature, handing a careful reader a reason to distrust every other label. "No private client data; the Blog Library is a real client's already-public content, labelled as such" would be both accurate and stronger.

Generating without autofilling returns someone else's post

```
const slug = currentPresetSlug || GENERATOR_PRESETS[0].slug;
```

Type your own topic, ignore Autofill, hit Generate, and the fallback fires. Verified: typing `My Own Unrelated Topic` produces `Done: Track AI Search Sales: E-Commerce Attribution Guide (2026)` added to your Blog Library. Nothing warns the visitor that their input was discarded, and the fields are neither required nor read-only.

The most natural first interaction on the page silently produces an unrelated article and asserts it as the result: the exact thing the preset-derivation design was built to prevent, one branch away from all that care. This is the only unlabelled untruth on the page.

Every citation link in every article is dead

The real article bodies are full of `<sup class="citation"><a href="#ref-1">` markers. The reader renders references as a plain `<ol>` with no `id` attributes, and no `id="ref-N"` target exists anywhere in the stored data either. Measured on one opened post: **21 citation links, 21 dead**.

This is the price of copying rendered HTML out of a site into a different template. It is a one-line fix, and worth doing, because the citations are the part of the demo that shows real, sourced work.

The countdown's interval leaks forever

`setTimeout(() => { libraryDemo = null; renderLibrary(); }, ...)` nulls the object that holds the interval handle, so `clearInterval` can never be called for it. Verified by counting DOM rebuilds: 4 per 3 seconds while the card lives, and **4 per 3 seconds after it expires**, permanently. Each tick rebuilds the entire fifteen-card grid, images and all, via `innerHTML`, to update one `mm:ss` string.

The replace path does clear both handles correctly; only the expiry path forgets. One line fixes it. The deeper point, that a countdown should update a text node rather than re-render a grid, is the clearest “I would do it differently” the codebase offers.

Smaller, but real

- **Credits go negative.** `creditsBalance -= 100` has no floor, in a demo whose subject is metering. Thirteen clicks reach `-100`. One `if` would turn a bug into a demonstration of the feature.
- **DEMO.facts is dead code**, never read anywhere, while the same three figures are hard-coded again in the footer of `index.html`. Two copies, one wired to nothing, in a project whose premise is traceable claims.
- **The derived keywords drop the subject.** The length filter deletes “Ads”, “SEO”, “AEO”, “CAC” and “vs”, so the Google Ads post’s keywords contain neither “ads” nor “AEO”; stopwords like “your” and “what” survive; and the punctuation strip misses `?`, shipping the literal keyword `2026?` on screen.
- **Two advertised capabilities are absent.** The hero names per-client editor roles and PDF export; neither appears anywhere in the tour.
- **The provenance names a different system.** `CREDITS.md` says `davonex.com` is a client of `blog-posting-pipeline`, a *separate* private project, while the page’s prose says only “the real production pipeline” inside a demo titled Agency Blog SaaS. A visitor will reasonably conclude these posts came from this product. Worth settling before an interview.
- **The escaping is theatre.** `escapeHtml` is applied diligently, but every string it touches is a hard-coded literal and no user input ever reaches the DOM. The page is safe because it has no untrusted input, not because it escapes. Good habit, accurate description needed.

6 The honest bottom line

What this demonstrates, and what it does not

It demonstrates that its author can build a small, clean, dependency-free static page, hold a strong self-imposed constraint (no network calls) that survives measurement at zero non-local requests, and think unusually carefully about how to label a fiction so nobody mistakes it for fact.

It contains **no evidence** about the real product: not whether generation works, not output quality, not prompting, not metering or refunds, not billing, not roles, not PDF export, not performance, not security, not scale, and not whether the tests it cites exist.

The strongest defensible sentence is: “This is a puppet, I built the puppet carefully, and I labelled it as a puppet.” Everything in this document supports that. Nothing in it supports a claim about the real system’s quality, and the demo is at its best when it does not try to make one.

Glossary

Static site

A site made of files served exactly as stored, with nothing generated per visitor. The opposite is a dynamic site, where a server program (Django, say) builds each page from a database. The real system is dynamic; this demo is static, which is why it can only imitate dynamism by scripting it in the browser.

DOM

The browser’s live in-memory tree of the page. HTML is the text you wrote; the DOM is the object the browser built from it and actually draws. JavaScript changes the page by changing the DOM.

innerHTML

The blunt way to change the DOM: assign a string of HTML and the browser discards everything inside the element and re-parses it. Simple to write; it destroys and recreates elements every time, and it executes whatever markup the string contains.

Event delegation

Attaching one listener to a container instead of one per child, then using `closest()` to find which child was hit. Necessary here because the cards are rebuilt every second and per-card listeners would die with them.

Slug

A lowercase, hyphenated, URL-safe form of a title. Here it is both a post’s identity inside the demo and the real path on the real site, which is how the provenance link is built by concatenation rather than stored.

setTimeout / setInterval

Ask the browser to call a function once after a delay, or repeatedly at an interval. Both return a handle; losing the handle means the timer can never be cancelled, which is exactly the bug above.

XSS and escaping

Cross-site scripting: if attacker-controlled text is pasted into HTML, the browser cannot tell it from your markup and will run any `<script>` in it. Escaping turns `<` into `<`; so it is drawn rather than parsed.

CSS custom properties

Variables like `--blue: #136BEE` declared on `:root` and read with `var(--blue)`, so the palette lives

in one block instead of being scattered as literal hex codes.

Netlify, `publish = "."`

A host that serves a git repository as a website. Most projects need a build command first; this one has no build, so the config says only “publish the folder as it is”.

PolyForm Strict 1.0.0

A source-available licence: read and evaluate the code, but do not use, modify or redistribute it. The right choice here, because the repository contains a real client’s copyrighted articles that are not the licensor’s to give away freely.